



Laboratory Activity No. 2

From Syntax to Semantics: Building a Grammar Checker and Evaluator

Name:	Section:
Rollin Manuel Furio	BSCS 4-1
Group No.	Date:
	September 21, 2026

I. The Grammar I Used

I used the corrected grammar given in Section II of the activity sheet:

```
<expr> -> <term> { ( + | - ) <term> }
<term> -> <factor> { ( * | / ) <factor> }
<factor> -> ( <expr> ) | <digit>
<digit> -> 0|1|2|3|4|5|6|7|8|9
```

I wrote the program in Python using a recursive-descent parser. There is one function for each rule: `expr()`, `term()` and `factor()`. Each function does exactly what its rule says, so the code is easy to compare with the grammar.

II. Files I Am Submitting

- `syntax_semantics.py` — my source code, with comments
- `Flowchart.drawio.png` — my flowchart
- `Output Log.png` — the output of a full run, including all required test cases
- This document — my answers to the reflection questions

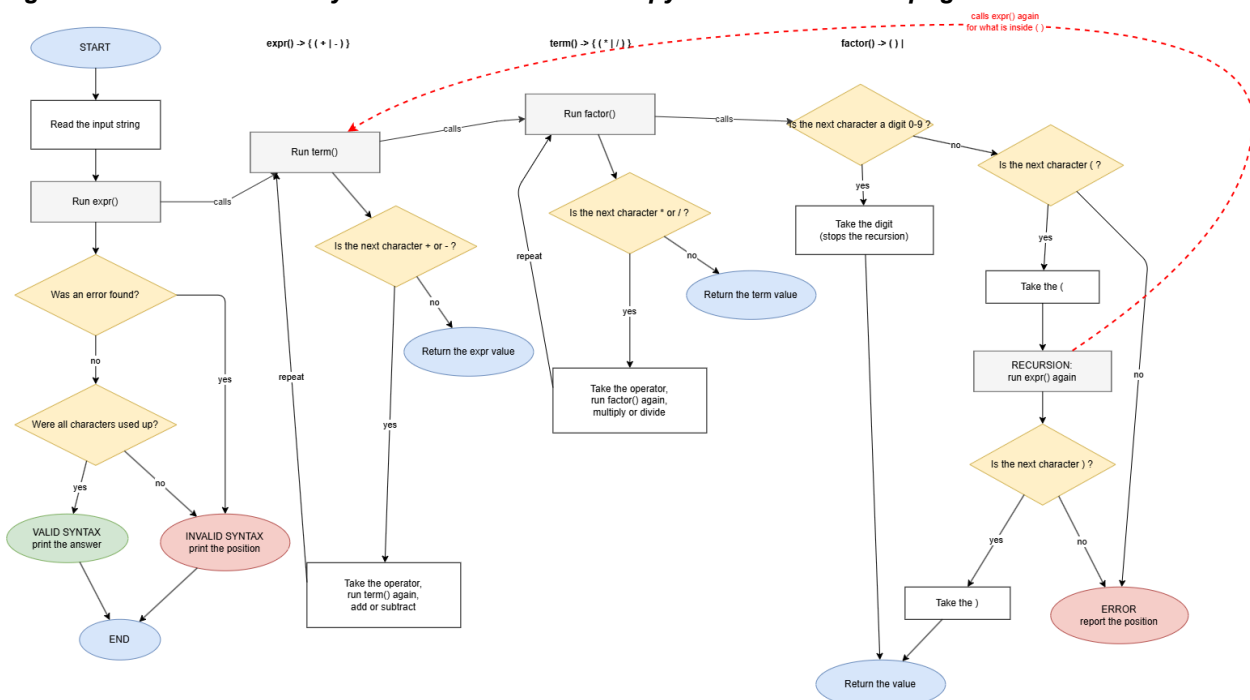
III. Part 1. Flowchart

The flowchart on the next page shows how the recognizer decides if an input is valid. The three grammar levels `expr()`, `term()` and `factor()` are drawn as separate boxes. The diamonds are the points where the next character decides what to do. The red dashed arrow shows the recursion, where `factor()` calls `expr()` again to handle whatever is inside the parentheses.





Figure 1. Flowchart of the syntax checker. Full size copy: [Flowchart.drawio.png](#)



IV. Part 2. Task A and Task B

Task A is the syntax check. The parser reads the input one character at a time and follows the grammar rules. If something does not fit like the extra + in 3++4, or the missing) in (3+4 it raises a BadSyntax error and the program prints "Invalid syntax" together with the position where it stopped.

Task B is the evaluation. It computes the answer. The important part is that the program only prints a result if Task A finished without raising an error. So an invalid expression never gets an answer printed for it. That is the difference between syntax and semantics in this program: first check that it is written correctly, then work out what it means.

V. Stretch Requirement: The Ambiguous Grammar

The ambiguous rule is:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid \langle \text{digit} \rangle$

This rule does not say which part to solve first, so $2+3*4$ can be read in two ways. My program shows both:

Ambiguous grammar (left to right): 20 $\rightarrow$ reads it as $(2+3)*4$
Corrected grammar (this program): 14 $\rightarrow$ reads it as $2+(3*4)$

The grammar in Section II fixes this by putting $\langle \text{term} \rangle$ underneath $\langle \text{expr} \rangle$. Because $\text{expr}()$ has to call $\text{term}()$ before it can handle a + or -, all the * and / are already finished by the time addition happens. That gives * and / higher precedence automatically. The { } repetition also works from left to right, which is why $9-4-2$ means $(9-4)-2$ and gives 3. With precedence and associativity both settled, there is only one way to read any expression, so $2+3*4$ can only be 14.





VI. Required Test Cases

Input	Expected Syntax	Expected Value	What My Program Printed	Match?
3+4*2	Valid	11	Valid, Result: 11	Yes
(3+4)*2	Valid	14	Valid, Result: 14	Yes
8/2-1	Valid	3	Valid, Result: 3	Yes
3++4	Invalid	—	Invalid, error at position 2	Yes
(3+4	Invalid	—	Invalid, error at position 4	Yes
2+3*4	Valid	14	Valid, Result: 14	Yes

All six cases matched what was expected. I also tested a few extra ones, which are in Output Log.png: 9-4-2 = 3 and 8/2*3 = 12 for left-to-right order, (1+2)*(3+4) = 21 for nested parentheses, 5/0 for division by zero, and 12 which is rejected because the grammar only allows single digits.

VII. Reflection Questions

1. In your own words, what is the difference between what your Task A code checks and what your Task B code checks?

Task A only checks if the expression is written correctly. It does not care about the answer. It reads the characters and checks them against the rules of <expr>, <term> and <factor>. If it finds something that does not belong, like the second + in 3++4, it stops and reports the position.

Task B is the part that gets the answer. It does the actual adding, subtracting, multiplying and dividing. It only runs if Task A did not find any error, so an invalid expression never gets a result. Syntax is about whether it is written properly, and semantics is about what it actually means.

2. Point to the exact place in your code where recursion happens, and explain which grammar rule it corresponds to.

The recursion is inside factor(), on the line marked *** RECURSION IS HERE ***. After factor() takes an opening parenthesis, it calls self.expr() to handle everything inside the parentheses.

This matches the rule <factor> -> (<expr>). factor() is the last function in the chain, but it calls expr(), which is the first one, so the parser goes back to the top and starts over for whatever is inside the brackets. For example, in (3+4)*2, factor() calls expr() to work out 3+4, gets 7 back, and then term() multiplies that 7 by 2.

The recursion stops because of the other option in the same rule, <factor> -> <digit>. When factor() sees a digit it just takes it and returns, without calling anything else. So every path eventually ends at a digit.





3. Explain, using your $2+3*4$ example, why an ambiguous grammar is a real problem and not just a theoretical one.

It is a real problem because the same input can give two different answers. With the ambiguous rule, $2+3*4$ can be read as $(2+3)*4$, which is 20, or as $2+(3*4)$, which is 14. Both readings are allowed by the grammar, so neither one is wrong according to the rules.

My program prints both results so you can see the difference. 20 and 14 are not close to each other — it is a completely different answer for the same input. If this happened in a real programming language, the same code could give different results on different compilers, and there would be no way to say which one is correct because the grammar allows both.

That is why the fix has to be in the grammar and not just in the code. The corrected grammar in Section II only allows one reading, so 14 is the only possible answer.

4. If you were to add a new operator (for example, exponentiation $^$) to this grammar, where in the `expr/term/factor` hierarchy would it need to go, and why?

Exponentiation is stronger than multiplication and division, so it needs its own level between `<term>` and `<factor>`. The grammar would become:

```
<expr> -> <term> { ( + | - ) <term> }
<term> -> <power> { ( * | / ) <power> }
<power> -> <factor> [ ^ <power> ]
<factor> -> ( <expr> ) | <digit>
```

In the code I would add a `power()` function, then change `term()` so that it calls `power()` instead of `factor()`, and `power()` would call `factor()`. Putting it there is what makes it work: since `term()` now has to go through `power()` first, any $^$ is done before the $*$ or $/$ around it. So $2*3^2$ would be $2*(3^2) = 18$ and not $(2*3)^2 = 36$.

One thing is different from the other operators though. Exponentiation goes right to left, so 2^3^2 means $2^(3^2) = 512$, not $(2^3)^2 = 64$. That is why `power()` has `<power>` on its right side instead of using the `{ }` loop that `expr()` and `term()` use. Calling itself on the right makes it build from the right, which gives the right-to-left order.

